

Assignment Four Report

Bizhe Bai, Muzhi Lyu

December, 2020

Kernel 1 & Kernel 2 – Single Pixel per Thread

In the first two kernels, we implemented the discrete Laplacian filter on GPU. The first few steps after the function is called involves allocating memory on devices and copying the input matrix, output matrix and filter to GPU memories. Besides all input and output, we allocated a maximum value and a minimum value on device, to help with the normalization later.

For all of our kernels, we use a fixed block size of 512, and calculate block number each time. Then inside our filter kernel, Kernel 1 and Kernel 2 functions similarly. Each thread calculated its thread id, and if the id is less than total number of pixels, the thread will get a pixel to work on. The thread will store the output in three locations: the output array, shared memory for maximum, and that for minimum. Then for the maximum/minimum reduction, our case is different from lecture examples, as we need two shared arrays for different reductions. Therefore we chose to double shared memory size, and use the second half of array for reduction to minimum. We used interleaved addressing for this part, and after the work is done for each thread, a local maximum will be the first element of shared array, and local minimum is the 512th.

To get the global maximum among blocks, we first planned to store result from each block, and do another reduction with this array, like what we did in Lab 10. However, this involves in writing another kernel and allocating more memories on device. Considering these cost, we turned to atomic operations, and let the first thread in each block calculate the global extreme values.

The normalization kernel will be called after while, which assigns pixels in the same way, and does the calculation with global maximum and minimum we got. Then we write the output matrix back to store the change we made, free allocated memories and exit.

Kernel 3 & Kernel 4 – Multiple Pixels per Thread

For these two kernels, we assigned multiple pixels to each thread in two ways. Ideally, each thread should work on 4 pixels, but some may have less work since the total number of threads is not always a multiple of 4.

The first approach is consecutive rows in Kernel 3. If a thread could work on 4 pixels, we will assign $512 \times 4 = 2048$ pixels to each block, and determine the grid size and block size beforehand. After launching the kernel, each thread will calculate its offset based on the global thread id, and start working if its offset is within the length of array (offset is less than length). Within a for loop, the thread will process until it reaches the end of array, or work on all 4 pixels is done. In the meanwhile, the maximum and minimum among these 4 pixels will be calculated and stored in one location of shared memory, determined by thread id within block. Then when the compiling is done, shared memory will still store an array of maximum values and an array of minimum values. The reduction technique we applied is the same as Kernel 1 and 2.

In Kernel 4, we implemented stride access, which is expected to be faster than the consecutive rows method. We still determine grid size by 2048 pixels per block, but for this time we distribute pixels differently. Each thread in kernel will calculate the total number of active threads, based on grid size and block size, and then work on the pixel assigned by its global thread id. Therefore all threads are working on the first part of input array together. Then it will jump k elements to its next offset, where k is the total number of active threads. This calculation will be done no more than 4 times. Finally, with the local maximum and minimum kept by each thread, we use similar reduction in shared memory to get global maximum and minimum.

Kernel 3 and 4 are similar, but even if the number of input pixels are the same, each thread in Kernel 3 and 4 has different amount of work. For example, if we have 8 pixels, then thread 0 and thread 1 in kernel 3 will work on 4 pixels, while the first 8 threads in Kernel 4 will calculate one pixel each.

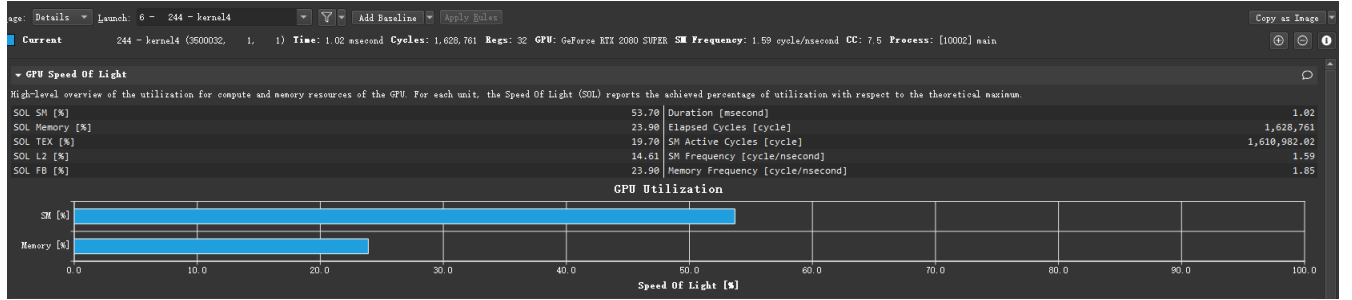


Figure 1:

Kernel 5,custom access

The main focus of our optimization is on memory transferring. Because we see from Nvidia profiling (1) .The peak memory bandwidth is far from 100%. So ,we create a pinned memory "input" in order to transfer data faster to GPU(devices) .As the output of the program shows, the transfer in time is decreased.Also ,we use some technique such as constant memory .The disadvantages and attempt will discussed in part Analysis.

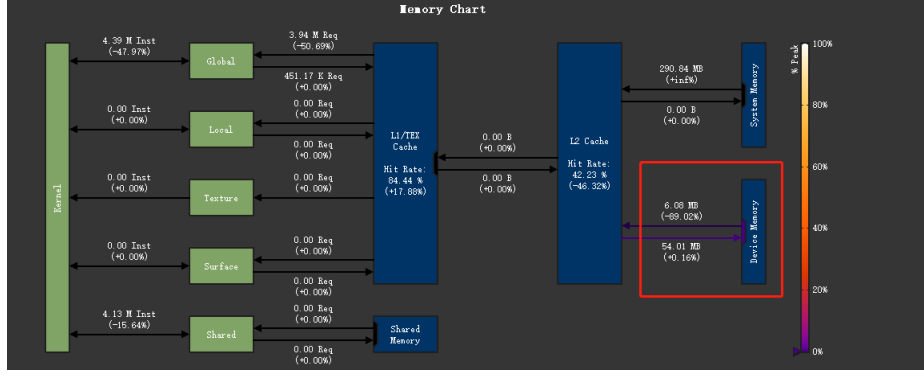


Figure 2:

	Sectors	% Peak	Bytes	Throughput
Load	199,332 (-89.02%)	0.05 (-99.55%)	6,378,624 (-89.02%)	266,796,942.97 (-99.53%)
Store	1,769,705 (+0.16%)	0.49 (-95.86%)	56,630,560 (+0.16%)	2,368,670,780.19 (-95.75%)
Total	1,969,037 (-45.04%)	0.54 (-97.73%)	63,009,184 (-45.04%)	2,635,467,723.15 (-97.67%)

Figure 3:

Analysis

1 First of all , the optimization is for the memory transfer brought the computational latency and memory transfer out latency .We believe that is cause by the real disk memory is been pinned ,thus, other processes in the application would have bad influence .This point is also mention in (1).The second attempt is to make output array also paged-locked(or pinned).However , it cause huge computational cost raise .After we searched for resources , this situation is because the data have been paged-locked on device could be CPU uncached , thus whenever we have a computation on the data .There is a huge data transfer overhead from device to host.

2 For our current attempt , the transfer in time have been reduce ,which showed in figure 2.The red box shows that the communication between system memory and device L2 cache is increased a lot compared with kernel 4.As well in figure 3 , this store and load instructions is reduced a lot compared to kernel 4.

For the negative side of our kernel .As figure 4 shows , the L1 tex requests is much fewer .This is not a good pattern since we need try to use data locally as much as possible .

	Instructions	L1/TEX Requests
Global Load Cached	3,936,204 (-50.69%)	3,936,204 (-50.69%)
Global Load Uncached	-	-
Local Load Cached	0 (+0.00%)	0 (+0.00%)
Local Load Uncached	-	-
Surface Load	0 (+0.00%)	0 (+0.00%)
Texture Load	0 (+0.00%)	0 (+0.00%)
Global Store	437,500 (+0.00%)	437,500 (+0.00%)
Local Store	0 (+0.00%)	0 (+0.00%)
Surface Store	0 (+0.00%)	0 (+0.00%)
Global Reduction	13,672 (+0.00%)	13,672 (+0.00%)
Surface Reduction	0 (+0.00%)	0 (+0.00%)
Global Atomic	0 (+0.00%)	0 (+0.00%)
Global Atomic Cas	0 (+0.00%)	0 (+0.00%)
Surface Atomic	0 (+0.00%)	0 (+0.00%)
Surface Atomic Cas	0 (+0.00%)	0 (+0.00%)
Loads	3,936,204 (-50.69%)	3,936,204 (-50.69%)

Figure 4:

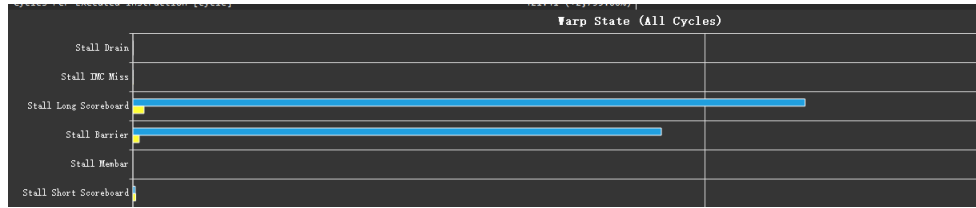


Figure 5:

3 For the warps . as figure 5 shows . Our warps have been barrierd and waiting for scoreboard dependency much more than kernel4's .The barrier waiting usually caused by code divergent before barrier . Since the loading data from CPU is more unpredictable than loading data from devices memory .This could make sense. All in all , the optimization for memory transfer might not be a good idea compared with optimize for computation. Thanks.
 (1)<https://forums.developer.nvidia.com/t/pinned-memory-slows-cpu-computation/40880>